

Хранилище (Store) в Nuxt.js

Что такое Store в Nuxt.js?

Store (хранилище) в Nuxt.js основано на Vuex — библиотеке управления состоянием для Vue.js. Оно позволяет централизованно хранить и управлять состоянием приложения, что особенно важно для сложных приложений с множеством компонентов.

Автоматическая интеграция Vuex

Nuxt.js автоматически интегрирует Vuex, если в директории `store/` есть файлы `.js`, `.ts` или `.mjs`. Это означает, что вам не нужно вручную настраивать Vuex — просто создайте нужные файлы, и Nuxt сделает все остальное.

Структура Store

Классический режим (Nuxt 2)

```
store/
--| index.js          # Корневой модуль хранилища
--| users.js          # Модуль users
--| products/
----| index.js        # Модуль products
----| mutations.js    # Мутации модуля products
----| actions.js      # Действия модуля products
```

Пример корневого модуля

```
// store/index.js
export const state = () => ({
  counter: 0
})

export const mutations = {
  increment(state) {
    state.counter++
  }
}

export const actions = {
  async fetchData({ commit }) {
    const data = await this.$axios.$get('/api/data')
    commit('setData', data)
  }
}
```

```
export const getters = {
  getCounter: state => state.counter
}
```

Пример модуля

```
// store/users.js
export const state = () => ({
  list: []
})

export const mutations = {
  add(state, user) {
    state.list.push(user)
  }
}

export const actions = {
  async fetchUsers({ commit }) {
    const users = await this.$axios.$get('/api/users')
    commit('setUsers', users)
  }
}
```

Использование Store в компонентах

```
<template>
  <div>
    <p>Счетчик: {{ $store.state.counter }}</p>
    <p>Через геттер: {{ $store.getters.getCounter }}</p>
    <button @click="$store.commit('increment')">Увеличить</button>
    <button @click="$store.dispatch('fetchData')">Загрузить данные</button>
  </div>
</template>
```

Модули с пространством имен

Все модули в директории `store/` автоматически получают пространство имен (namespaced).

```
<template>
  <div>
    <p>Пользователи: {{ $store.state.users.list }}</p>
    <button @click="$store.dispatch('users/fetchUsers')">Загрузить
    пользователей</button>
  </div>
</template>
```

```
</div>
</template>
```

Store в Nuxt 3

В Nuxt 3 рекомендуется использовать Pinia вместо Vuex:

```
npm install pinia @pinia/nuxt
```

```
// nuxt.config.ts
export default defineNuxtConfig({
  modules: ['@pinia/nuxt']
})
```

Создание хранилища с Pinia

```
// stores/counter.ts
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', {
  state: () => ({
    count: 0
  }),
  getters: {
    doubleCount: (state) => state.count * 2
  },
  actions: {
    increment() {
      this.count++
    }
  }
})
```

Использование Pinia в компонентах

```
<script setup>
import { useCounterStore } from '~/stores/counter'

const counter = useCounterStore()
</script>

<template>
  <div>
```

```
<p>Счетчик: {{ counter.count }}</p>
<p>Удвоенный счетчик: {{ counter.doubleCount }}</p>
<button @click="counter.increment()">Увеличить</button>
</div>
</template>
```

Заключение

Хранилище в Nuxt.js — это мощный инструмент для управления состоянием приложения. Автоматическая интеграция Vuex в Nuxt 2 и поддержка Pinia в Nuxt 3 делают работу с состоянием приложения простой и удобной. Правильное использование хранилища позволяет создавать масштабируемые и поддерживаемые приложения с предсказуемым потоком данных.